Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution affiliated to Anna University)

Degree & Branch	B.E Computer Science & Engineering	**Semester**	VI
Subject Code & Name	UCS2612 – Machine Learning Laboratory		
Academic Year	2025–2026 (Even)	**Batch**	2023–2027
Name	Mehanth T	**Register No.**	3122235001080

Experiment 5

Perceptron vs Multilayer Perceptron with Hyperparameter Tuning

Objective

To implement and compare the performance of:

- **Model A:** Single-Layer Perceptron Learning Algorithm (PLA)
- **Model B:** Multilayer Perceptron (MLP) with hidden layers and nonlinear activation functions

Students must select, tune, and justify hyperparameters such as learning rate, activation function, optimizer, batch size, and number of hidden layers.

Dataset

English Handwritten Characters Dataset

Download Link: https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset

- Total samples: 3,410
- Number of classes: 62 (0–9, A–Z, a–z)
- Image type: Grayscale

Theory

Perceptron Learning Algorithm (PLA)

The Perceptron is a linear classifier that uses a step activation function to separate data into two classes.

$$w_{t+1} = w_t + \eta(y - \hat{y})x$$

- Uses a step activation function
- Converges only for linearly separable data

- Extended to multi-class problems using One-vs-Rest strategy

Limitation: PLA cannot learn nonlinear decision boundaries required for handwritten character recognition.

Multilayer Perceptron (MLP)

An MLP consists of one or more hidden layers with nonlinear activation functions.

- Activation functions: ReLU, Sigmoid, Tanh
- Loss function: Categorical Cross-Entropy
- Optimizers: SGD, Adam
- Learns nonlinear decision boundaries using backpropagation

Steps for Implementation

1. Preprocess the dataset (resize, flatten, normalize, **StandardScaler**).
2. Implement **PLA** from scratch:
    - Use step activation function.
    - Apply perceptron weight update rule.
    - Extend PLA to multi-class classification using One-vs-Rest.
3. Implement **MLP**:
    - Select number of hidden layers and neurons.
    - Choose activation functions.
    - Train using backpropagation.
4. Perform **hyperparameter tuning**:
    - Learning rate
    - Batch size
    - Optimizer
    - Activation function
5. Compare PLA and MLP using evaluation metrics.

Performance Evaluation

Evaluation Metrics

The MLP model significantly outperformed the PLA model. The confusion matrix and loss curves (generated in plots/) demonstrate that the MLP effectively learned features, whereas the PLA struggled with the non-linear complexity of the image data. The critical factor for MLP success was the application of **StandardScaler** to normalize input features to zero mean and unit variance.

Table 1: Overall Performance Comparison between PLA and MLP

Model	**Accuracy (%)**	**Precision**	**Recall**	**F1-score**
PLA (OvR)	9.24%	0.09	0.09	0.09
MLP (Tuned)	41.50%	0.43	0.41	0.41

Table 2: Hyperparameter Tuning Results for MLP

Hidden Layers	**Activation**	**Optimizer**	**Learning Rate**	**Batch Size**	**Accuracy (%)**
1 (128)	ReLU	SGD	0.01	32	¡ 30%
2 (256–128)	ReLU	Adam	0.001	64	**41.50%**
3 (512–256–128)	Tanh	Adam	0.0005	64	¡ 30%

Table 3: Training Convergence Comparison

Model	**Epochs**	**Final Training Loss**	**Convergence Behavior**
PLA	10	N/A	Poor Convergence
MLP (Tuned)	300	Low	Stable convergence (Early Stopping)

A/B Experiment Comparison

- **Final Tuned Hyperparameters:** Hidden Layers: (256, 128), Activation: ReLU, Optimizer: Adam, Learning Rate: 0.001, Batch Size: 64.
- **Strengths/Weaknesses:** PLA is simple but fails on complex image data (9% accuracy). MLP is computationally intensive but achieves much higher accuracy (41.5%) by capturing non-linear relationships.
- **Tuning Effect:** The 'Adam' optimizer and 2-layer architecture provided the best stability and performance compared to SGD and single-layer configurations.

Observation Questions

- **Why does PLA underperform compared to MLP?** PLA is a linear classifier and cannot capture the complex, non-linear pixel relationships inherent in handwritten characters. MLP uses non-linear activation functions (ReLU) to model these boundaries.
- **Which hyperparameter had the most impact on MLP performance?** Feature Scaling (StandardScaler) was the most critical preprocessing step. Architecturally, the number of hidden layers and the choice of 'Adam' optimizer were significant.
- **How does optimizer choice affect convergence?** Adam (Adaptive Moment Estimation) generally converges faster and more reliably on sparse/complex data like images compared to standard SGD, which can get stuck in local minima or oscillate.

- **Does increasing hidden layers always improve performance?** No. While more layers add capacity, they also increase the risk of overfitting and training difficulty (vanishing/exploding gradients). In this experiment, 2 layers worked better than 3 or 1.
- **How can overfitting be detected and mitigated?** Overfitting is detected when training accuracy is high but validation/test accuracy is low. It can be mitigated using regularization (L2), Dropout, or Early Stopping (used in this experiment).